Resolución de la Parte Teórica: Modo Protegido

[Fuster Lucio][1]

[1] UTN, FRA, Avellaneda, Buenos Aires, Argentina · ORCID: N/A

Resumen

Resolución del cuestionario teórico correspondiente a la Evaluación Globalizadora B, abordando los mecanismos de protección de memoria del 80x386, descriptores, el Segmento de Estado de Tarea (TSS) y el proceso de paginado.

Palabras clave: Modo Protegido; 80x386; GDT; LDT; TSS; Paginado; UTN FRA

1. Mecanismos de Protección de acceso al Segmento en Modo Protegido

a. Diagrama del mecanismo de protección de memoria del 80x386

En Modo Protegido, el procesador no accede directamente a la dirección física a través de los registros de segmento. En su lugar, el valor cargado en un registro de segmento (CS, DS, SS, ES, FS, GS) se denomina **Selector**. Este selector ya no contiene la dirección base, sino un índice que apunta a una tabla de descriptores ubicada en memoria RAM. Es el **descriptor** (y no el programa) el que contiene la base, el límite y los atributos de protección del segmento; por eso el programa no puede acceder directamente al segmento: siempre debe pasar por la verificación del procesador.

La Figura 1 muestra la visión general de la traducción de direcciones, la Figura 2 detalla cómo el selector localiza el descriptor y la Figura 3 las verificaciones de protección que se hacen antes de generar la dirección lineal.

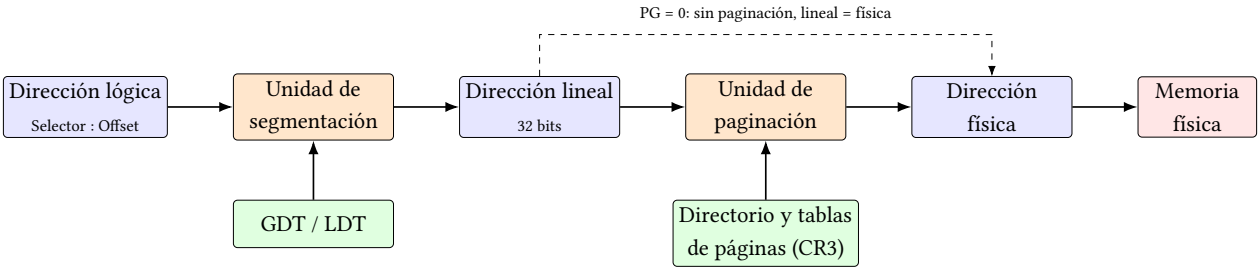


Figura 1: Traducción de dirección lógica a física en el 80x386.

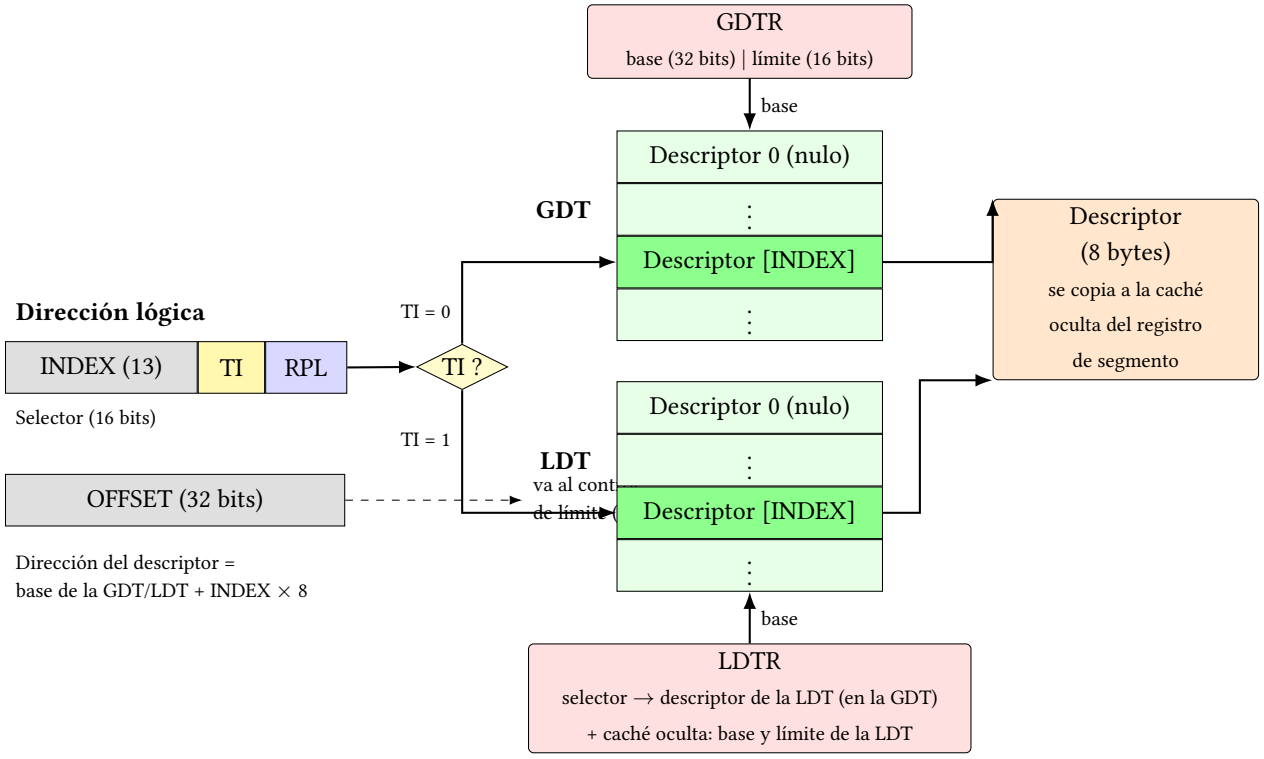


Figura 2: Localización del descriptor a partir del selector (bit TI).

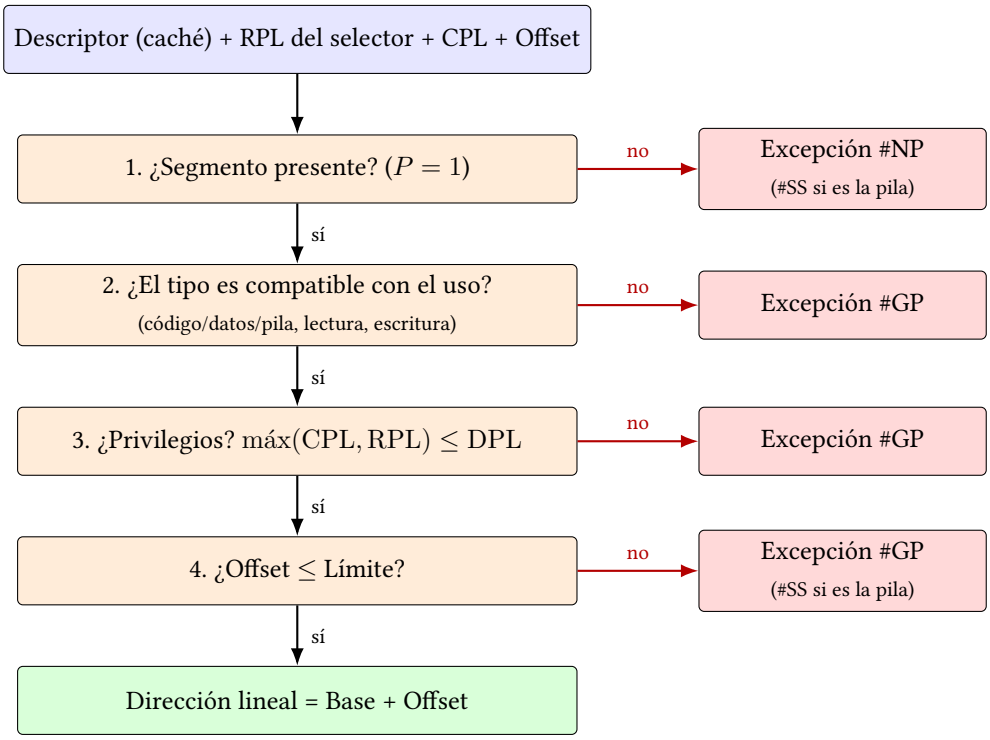


Figura 3: Verificaciones de protección previas a la generación de la dirección lineal.

Interpretación del registro de segmento según el bit TI: El Selector está compuesto por tres campos: **INDEX** (bits 15-3), **TI** (bit 2) y **RPL** (bits 1-0).

- **Si TI = 0:** El índice apunta a la **GDT** (Global Descriptor Table). Esta tabla es global para todo el sistema y contiene descriptores de segmentos de código, datos y sistema comunes a todas las tareas. Su ubicación la da el registro GDTR.
- **Si TI = 1:** El índice apunta a la **LDT** (Local Descriptor Table). Esta tabla es local para cada tarea y contiene los descriptores de los segmentos exclusivos de esa tarea. Para acceder a la LDT, el procesador utiliza el registro LDTR, que a su vez es cargado mediante un selector que apunta a un descriptor de LDT en la GDT.

En ambos casos, el descriptor buscado se encuentra en la posición base de la tabla $+$ INDEX $\times$ 8, ya que cada descriptor ocupa 8 bytes.

b. Condiciones para acceder al segmento y relación entre RPL y DPL

Para que un selector pueda acceder a un segmento, el procesador realiza rigurosas comprobaciones. El acceso se concede si se cumplen las siguientes condiciones:

1. El descriptor debe estar marcado como **Presente (P = 1)**; si no, se genera #NP (o #SS para la pila).
2. El **tipo** de segmento debe coincidir con el uso que se le quiere dar (por ejemplo, no se puede escribir en un segmento de código ni cargar en CS un segmento de datos).
3. El desplazamiento (Offset) debe estar dentro del **Límite** del segmento.
4. **Validación de Privilegios:** Aquí intervienen el **CPL** (Current Privilege Level, es decir, el RPL del selector cargado en CS), el **RPL** (Requested Privilege Level, contenido en el selector) y el **DPL** (Descriptor Privilege Level, contenido en el descriptor).

Si alguna condición falla, el procesador genera una excepción (#GP, #NP o #SS) y el acceso no se realiza.

Relación RPL y DPL: los niveles de privilegio van de 0 (máximo privilegio, kernel) a 3 (mínimo, usuario). El RPL permite que un programa privilegiado que actúa en nombre de otro menos privilegiado *degrade* voluntariamente su nivel efectivo, de modo que el acceso se evalúe con el peor de los dos niveles.

- **Para segmentos de datos:** El acceso se permite si $\max(\mathrm{CPL}, \mathrm{RPL}) \leq \mathrm{DPL}$. Es decir, el nivel de privilegio del descriptor debe ser numéricamente mayor o igual (menos privilegiado) que el nivel del procesador (CPL) y que el nivel solicitado (RPL). Esto evita que un código de usuario (CPL=3) manipule selectores para acceder a datos del sistema (DPL=0).
- **Para el segmento de pila (SS):** se exige $\mathrm{CPL} = \mathrm{RPL} = \mathrm{DPL}$.
- **Para segmentos de código no conforming** (transferencia con JMP/CALL directo): se exige $\mathrm{DPL} = \mathrm{CPL}$ y $\mathrm{RPL} \leq \mathrm{CPL}$. Para un código *conforming* basta con $\mathrm{DPL} \leq \mathrm{CPL}$ y el CPL no cambia.

c. Explicación de registros y bits

La Figura 4 muestra dónde se ubican los bits G, D, P, S, E y A dentro del descriptor de 8 bytes.

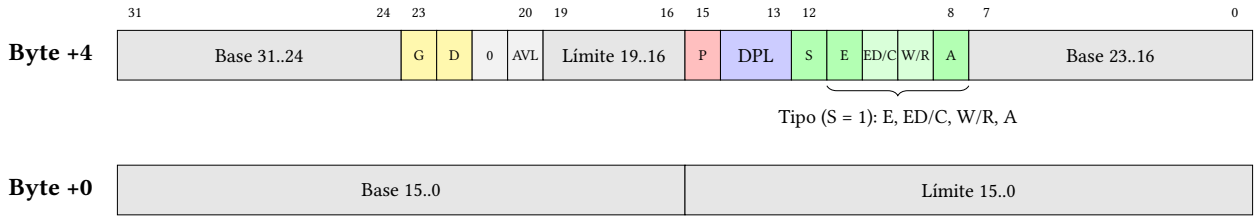


Figura 4: Formato del descriptor de segmento de 8 bytes (S = 1) y ubicación de los bits G, D, P, S, E, A.

- **GDTL (Global Descriptor Table Limit):** Campo de 16 bits del registro GDTR que indica el tamaño límite de la GDT, expresado en *bytes* (offset del último byte válido). La cantidad de descriptores es $(\text{GDTL} + 1)/8$; con el máximo de FFFFh caben 8192 descriptores.
- **GDTR (Global Descriptor Table Register):** Registro de 48 bits que almacena la dirección base lineal (32 bits) y el límite (16 bits) de la GDT en memoria RAM. Se carga con la instrucción LGDT.
- **RPL (Requested Privilege Level):** Bits 0-1 del selector. Indica el nivel de privilegio que el programa *solicita* para acceder al segmento.
- **LDTR (Local Descriptor Table Register):** Registro que almacena el selector del descriptor de la LDT actual (ubicado en la GDT) y mantiene una caché oculta con la base y el límite de la LDT de la tarea en ejecución. Se carga con LLDT.
- **Bit G (Granularity):** Si es 0, el límite se interpreta en bytes (máximo 1 MB). Si es 1, el límite se interpreta en páginas de 4 KB (multiplicado por 4096, máximo 4 GB).
- **Bit D (Default Operation Size):** Si es 0, el segmento de código opera por defecto en 16 bits. Si es 1, opera en 32 bits (afecta a operandos y direcciones). En segmentos de pila este bit se llama B y define si se usa SP o ESP.
- **Bit P (Present):** Indica si el segmento está cargado en memoria física. Si es 0, cualquier acceso genera una excepción de "Segment Not Present" (#NP).
- **Bit S (System):** Si es 0, el descriptor es de Sistema (TSS, LDT, compuertas). Si es 1, el descriptor es de Código o Datos.
- **Bit E (Executable):** Bit 3 del campo Tipo (válido con S = 1). Si es 1, el segmento es de Código (ejecutable). Si es 0, es de Datos.
- **Bit A (Accessed):** El procesador lo pone en 1 automáticamente cuando el selector del segmento es cargado en un registro de segmento. El SO lo usa para saber qué segmentos han sido utilizados y cuáles pueden ser liberados.

d. Práctica: Diagrama interpretando el selector CS = A3FF

Dado el selector A3FF en hexadecimal, lo convertimos a binario para analizar sus campos: A3FF = 1010 0011 1111 1111

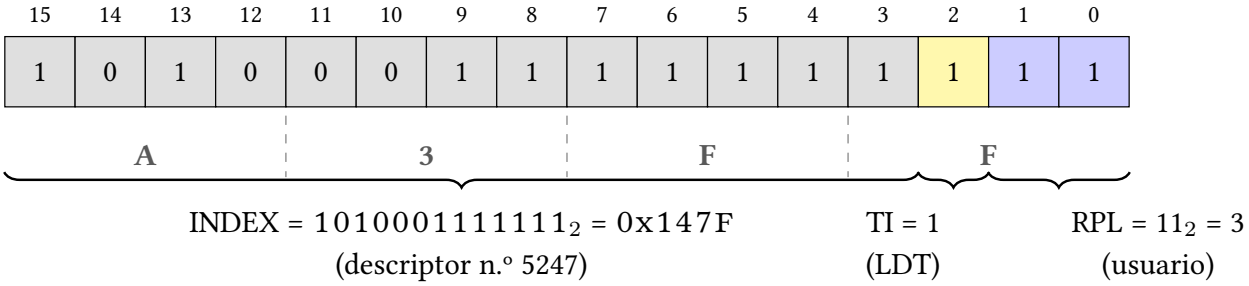


Figura 5: Interpretación del Selector CS = A3FF.

Interpretación:

- **Index = 0x147F (5247):** El procesador buscará el descriptor número 0x147F en la tabla indicada. Su posición dentro de la tabla es $\text{INDEX} \times 8 = \texttt{0xA3F8}$ bytes a partir de la base de la tabla; para que el acceso sea válido, el límite de esa tabla debe ser al menos 0xA3FF.
- **TI = 1:** El bit TI está en 1, por lo tanto, el índice apunta a la **LDT** (Local Descriptor Table), cuya base y límite están en el LDTR.
- **RPL = 3:** El nivel de privilegio solicitado es 3 (el más bajo, típico de modo usuario). Como el selector está en CS, su RPL es además el CPL actual: el código se ejecuta con privilegio de usuario, y el descriptor de código elegido debe tener DPL = 3.

2. Descriptores de Segmento y Descriptores de Sistema

a. Tablas de descriptores en la arquitectura 80x386

En la arquitectura IA-32 (80x386), existen **3 tablas de descriptores** principales:

1. **GDT (Global Descriptor Table):** Es la tabla principal del sistema. Contiene descriptores de segmentos de código, datos y sistema que son compartidos por todas las tareas (código del kernel, datos globales, descriptores de LDT, TSS, etc.). Solo hay una GDT por sistema, y su primer descriptor (índice 0) es nulo.
2. **LDT (Local Descriptor Table):** Es una tabla opcional por cada tarea. Contiene descriptores de los segmentos exclusivos de esa tarea (código de la aplicación, datos locales, pila). Cada LDT está definida por un descriptor de sistema dentro de la GDT.
3. **IDT (Interrupt Descriptor Table):** Es la tabla de descriptores de interrupciones. Contiene hasta 256 entradas (puertas de interrupción, de trap o de tarea) que mapean los tipos de interrupción y excepciones a sus correspondientes rutinas de atención (handlers). Su ubicación la da el registro IDTR.

Nota: No debe confundirse "tabla de descriptores" (las estructuras GDT, LDT, IDT) con "descriptor" (la estructura de 8 bytes que describe un segmento o una compuerta).

b. El bit S como atributo del descriptor

El bit S (bit 12 del segundo dword del descriptor, ver Figura 4) define qué clase de objeto describe la entrada, y por lo tanto cómo se interpretan los 4 bits del campo Tipo:

- **S = 1:** el descriptor es un **descriptor de segmento** (de código o de datos). El campo Tipo contiene los bits E (ejecutable), ED/C (expand-down para datos, conforming para código), W/R (escribible para datos, legible para código) y A (accedido).
- **S = 0:** el descriptor es un **descriptor de sistema**. El campo Tipo selecciona qué objeto de sistema se describe.

Tipo	Descriptor de sistema (S = 0)	Dónde puede estar
1h / 3h	TSS de 16 bits (disponible / ocupado)	GDT
2h	LDT	GDT
4h	Call Gate de 16 bits	GDT, LDT
5h	Task Gate	GDT, LDT, IDT
6h / 7h	Interrupt / Trap Gate de 16 bits	IDT
9h / Bh	TSS de 32 bits (disponible / ocupado)	GDT
Ch	Call Gate de 32 bits	GDT, LDT
Eh / Fh	Interrupt / Trap Gate de 32 bits	IDT

Cuadro 1: Tipos de descriptores de sistema del 80x386.

Además, con S = 1 existen dos tipos de descriptores de segmento: **datos** (E = 0) y **código** (E = 1).

c. "Call via Gate"

Un **Call Gate** (compuerta de llamada) es un descriptor de sistema (tipo Ch en 32 bits) que permite que un programa de menor privilegio llame, de forma *controlada*, a código de mayor privilegio. En lugar de apuntar directamente al segmento de código destino (lo cual la protección impediría si el DPL es menor al CPL), el programa ejecuta un `CALL FAR selector:offset` donde el **selector apunta al Call Gate** y el offset de la instrucción es ignorado.

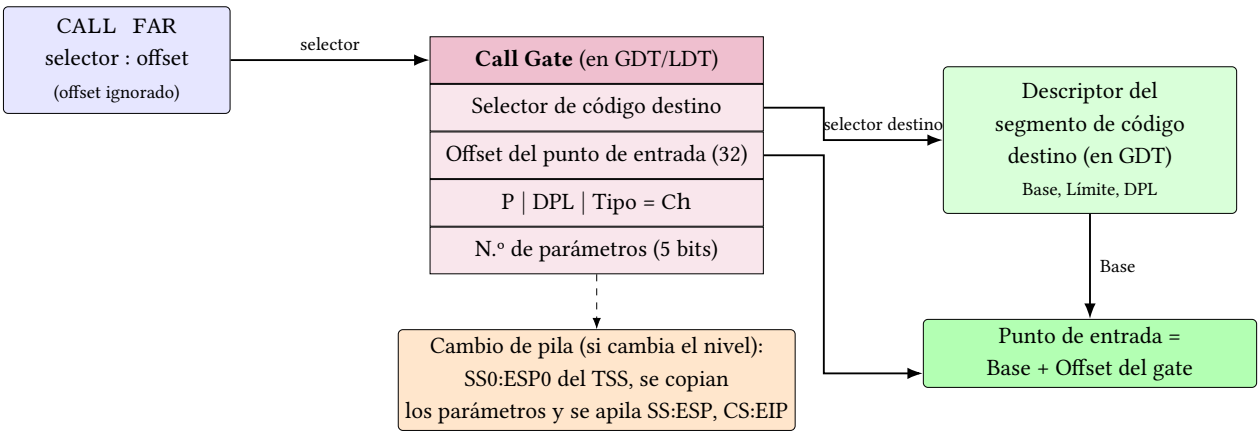


Figura 6: Funcionamiento de un Call via Gate.

Funcionamiento:

1. La instrucción CALL FAR carga un selector que apunta a un descriptor de Call Gate en la GDT o LDT.
2. El procesador verifica que el gate esté presente y que $\text{máx}(\text{CPL}, \text{RPL}) \leq \text{DPL}_{gate}$, es decir, que el llamador tenga privilegio suficiente *para usar la compuerta.*
3. Toma del gate el selector del segmento de código destino y verifica que su $\text{DPL} \leq \text{CPL}$ (solo se puede transferir a igual o mayor privilegio).
4. El punto de entrada es el **offset almacenado en el gate** (no el de la instrucción), por lo que el llamador solo puede entrar por las direcciones que el sistema operativo decidió exponer.
5. Si cambia el nivel de privilegio, el procesador toma la nueva pila (SS0:ESP0, SS1:ESP1 o SS2:ESP2) del TSS, copia desde la pila anterior la cantidad de parámetros indicada en el gate y guarda SS:ESP y CS:EIP anteriores. Un RET FAR deshace el proceso.

Así, las aplicaciones de usuario pueden invocar servicios del kernel sin poder saltar a cualquier dirección del código privilegiado.

3. Segmento de Estado de la Tarea (TSS)

a. Uso del TSS en la conmutación de tareas

El **TSS** (Task State Segment) es una estructura de datos en memoria utilizada por el procesador para almacenar el **contexto de ejecución** de una tarea, es decir, el valor de los registros internos del procesador al momento de suspenderla. Su propósito principal es permitir la **conmutación de tareas por hardware** (hardware multitasking). Cuando el procesador cambia de una tarea a otra, guarda automáticamente el estado de todos los registros del procesador (EIP, EFLAGS, registros de propósito general, registros de segmento, etc.) en el TSS de la tarea saliente, y luego carga el estado desde el TSS de la tarea entrante. Esto permite que el sistema operativo (a través de su scheduler) implemente multitarea de forma rápida y segura sin tener que guardar manualmente

cada registro en la pila. Además, el TSS aporta las pilas de los niveles de privilegio 0, 1 y 2 que se usan cuando una tarea transfiere el control a un nivel más privilegiado.

b. Funcionamiento y esquema del TSS

Cada TSS requiere un **descriptor de TSS** en la GDT (S = 0, tipo 9h si está disponible o Bh si está ocupado, bit B = Busy). El registro de tarea (TR) contiene el selector hacia dicho descriptor y mantiene una caché oculta con la base y el límite del TSS. El **límite mínimo** es **67h**, es decir que el tamaño mínimo del TSS es de **68h = 104 bytes**; si el límite es menor se genera la excepción #TS (tipo 0Ah). Si se usa el mapa de permisos de E/S, el TSS es más grande.

La conmutación se dispara con un CALL o JMP a un selector de TSS (o de Task Gate), con una interrupción o excepción manejada por una tarea, o con IRET si el flag NT (bit 14 de EFLAGS) es 1.

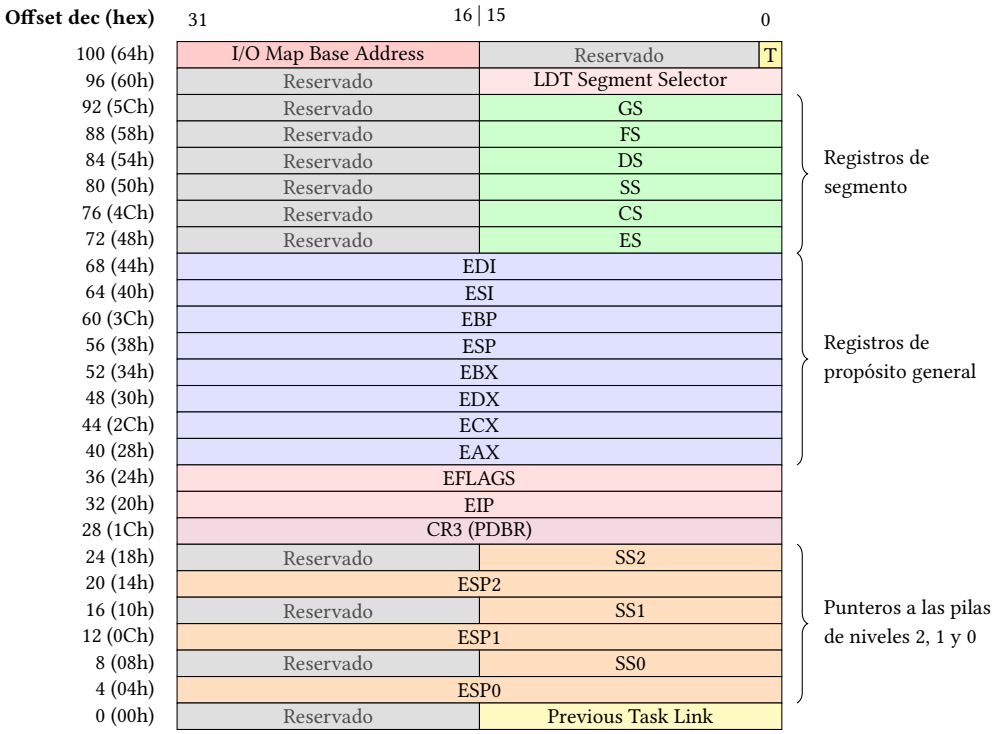


Figura 7: Esquema del Task State Segment (TSS) de 32 bits (tamaño mínimo 104 bytes). Los offsets se muestran en decimal y hexadecimal.

Campos clave:

- **Previous Task Link:** Almacena el selector del TSS de la tarea anterior (usado para tareas anidadas). Junto con el flag NT de EFLAGS permite que un IRET vuelva a la tarea previa.
- **SS0, ESP0, SS1, ESP1, SS2, ESP2:** Punteros a las pilas de los niveles de privilegio 0, 1 y 2. Cuando una tarea pasa a un nivel más privilegiado (ej: de CPL=3 a CPL=0), el procesador carga automáticamente la pila desde aquí. La pila de nivel 3 se encuentra en los registros SS:ESP.

- **CR3:** Dirección base del Directorio de Tablas de Páginas de la tarea.
- **EIP, EFLAGS, Registros de Propósito General y de Segmento:** Estado de la CPU.
- **LDT Segment Selector:** Selector de la LDT de la tarea.
- **T:** Si es 1, genera una excepción de Debug cada vez que se conmuta a esta tarea.
- **I/O Map Base Address:** Desplazamiento de 16 bits desde el inicio del TSS hasta el mapa de bits de permisos de E/S (para permitir o denegar acceso a puertos cuando CPL $>$ IOPL).

Cuando una tarea se despacha con CALL o una interrupción, el procesador anota el TR actual en el *Previous Task Link* y activa NT; el bit Busy del descriptor de la tarea previa permanece en 1 para evitar la reentrancia (un nuevo CALL a una tarea ocupada produce #GP).

4. Paginado

a. Concepto de paginado y relación con el segmentado

El **paginado** es un mecanismo de gestión de memoria que divide el espacio de direcciones lineales en bloques de tamaño fijo llamados **páginas** (generalmente de 4 KBytes). Su propósito principal es eliminar la fragmentación externa que produce el segmentado (cuyo tamaño es variable) y permitir la implementación de **memoria virtual**, mapeando páginas virtuales a marcos de página físicos no contiguos (y pudiendo llevar páginas poco usadas a disco).

Relación con el segmentado: En el 80x386, la segmentación y el paginado son mecanismos complementarios que actúan en serie.

1. La **Unidad de Segmentación** recibe la dirección lógica (Selector + Offset) y la traduce a una **dirección lineal** de 32 bits.
2. Si el paginado está desactivado (bit PG del CR0 = 0), la dirección lineal es igual a la dirección física.
3. Si el paginado está activado (bit PG = 1), la **Unidad de Paginación** toma la dirección lineal y la traduce a la **dirección física**.

b. Funcionamiento de la Unidad de Paginación

La unidad de paginación (parte de la MMU) traduce direcciones lineales de 32 bits a direcciones físicas utilizando un esquema de **paginación de dos niveles**:

1. La dirección lineal de 32 bits se divide en tres campos:
    - **Directorio (10 bits):** Índice dentro del Directorio de Tablas de Páginas (Page Directory), de 1024 entradas.
    - **Tabla (10 bits):** Índice dentro de la Tabla de Páginas (Page Table) seleccionada, también de 1024 entradas.

- **Offset (12 bits):** Desplazamiento dentro de la página física de 4 KB ($2^{12} = 4096$).

2. El registro **CR3** contiene la dirección física base del Directorio de Tablas de Páginas.
3. Los 10 bits del Directorio seleccionan una entrada (PDE - Page Directory Entry). Esta entrada apunta a la base física de una Tabla de Páginas.
4. Los 10 bits de la Tabla seleccionan una entrada (PTE - Page Table Entry). Esta entrada contiene la dirección base del marco de página físico (Page Frame).
5. Los 12 bits del Offset se suman a la base física del marco de página para obtener la dirección física final.

Cada PDE/PTE (4 bytes) incluye además bits de control, entre ellos P (presente), R/W (escritura), U/S (usuario/supervisor), A (accedido) y D (modificado). Si P = 0 o se viola un permiso, se genera la excepción #PF (Page Fault) y la dirección que la causó queda en CR2. Para no hacer dos accesos extra a memoria en cada referencia, el procesador guarda las traducciones recientes en la **TLB** (Translation Lookaside Buffer).

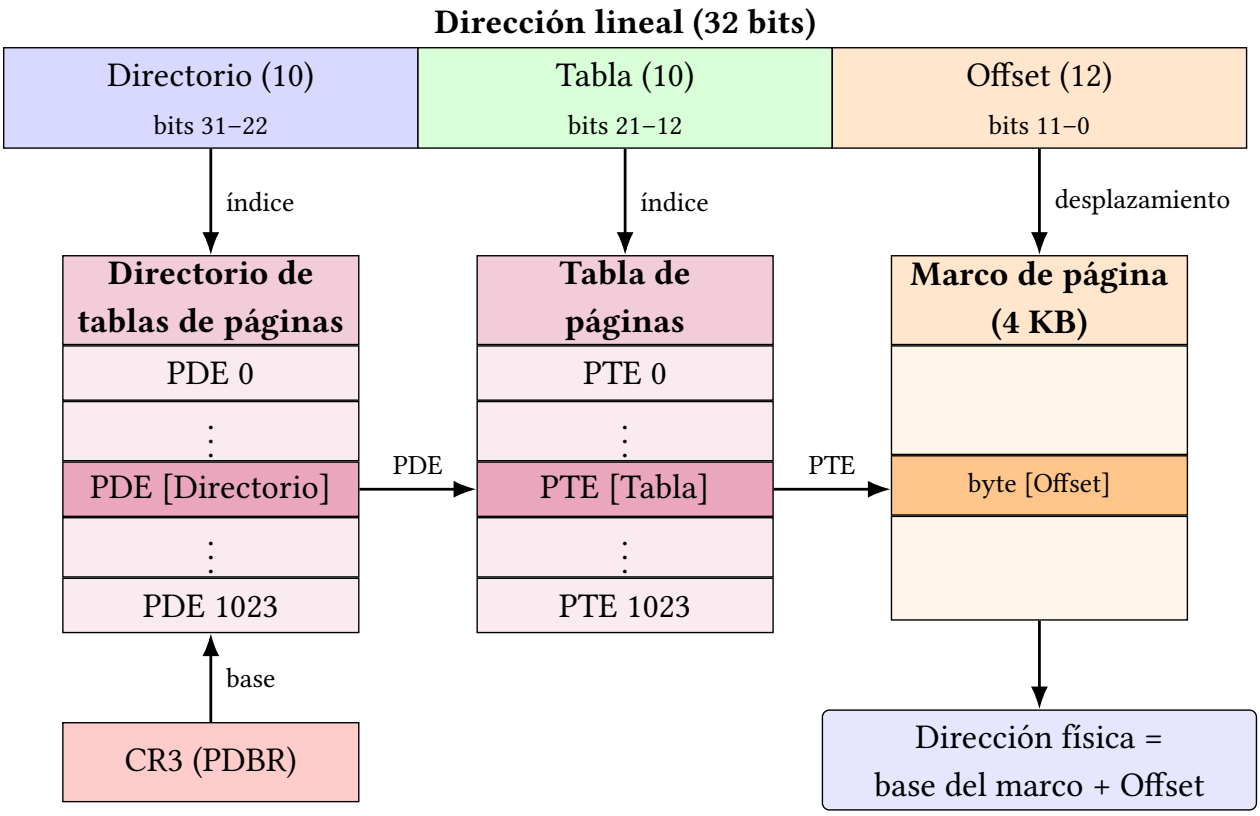


Figura 8: Esquema de traducción de paginación de dos niveles.